

https://moodle.joinville.udesc.br/pluginfile.php/429045/mod_resource/content/4/PAA_05.pdf

Projeto e Análise de Algoritmos

Algoritmos Eficientes de Ordenação:
Merge Sort, Quick Sort e ordenações lineares

André Tavares da Silva

andre.silva@udesc.br

Dividir e Conquistar

Desmembrar o problema original em vários subproblemas semelhantes, resolver os subproblemas (executando o mesmo processo recursivamente) e combinar as soluções.

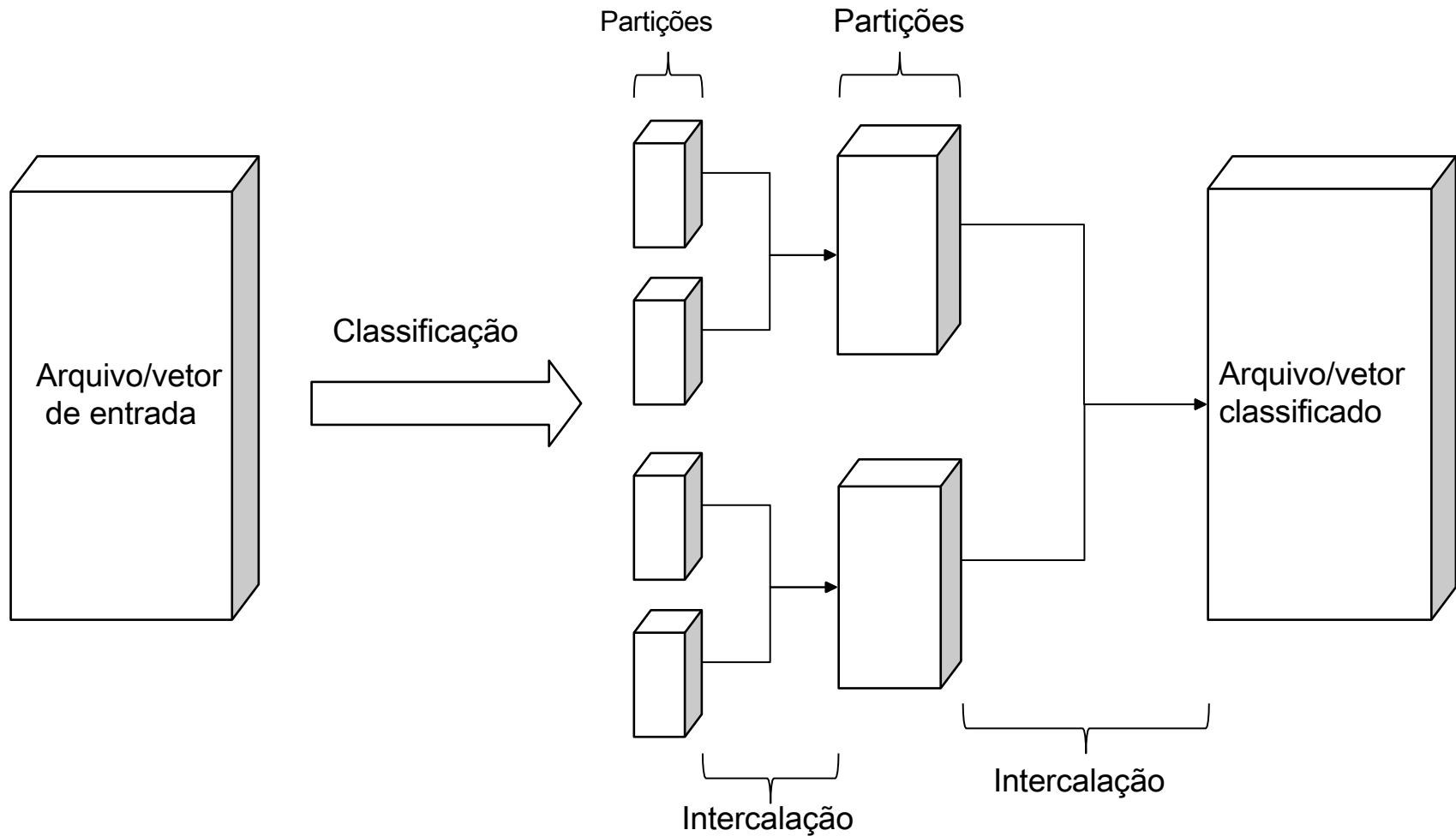
Merge Sort

Trata-se de um algoritmo recursivo que divide uma lista continuamente pela metade. Se a lista estiver vazia ou tiver um único elemento, ela está ordenada por definição (o caso base). Se a lista tiver mais de um elemento, dividimos a lista e invocamos recursivamente um Mergesort em ambas as metades.

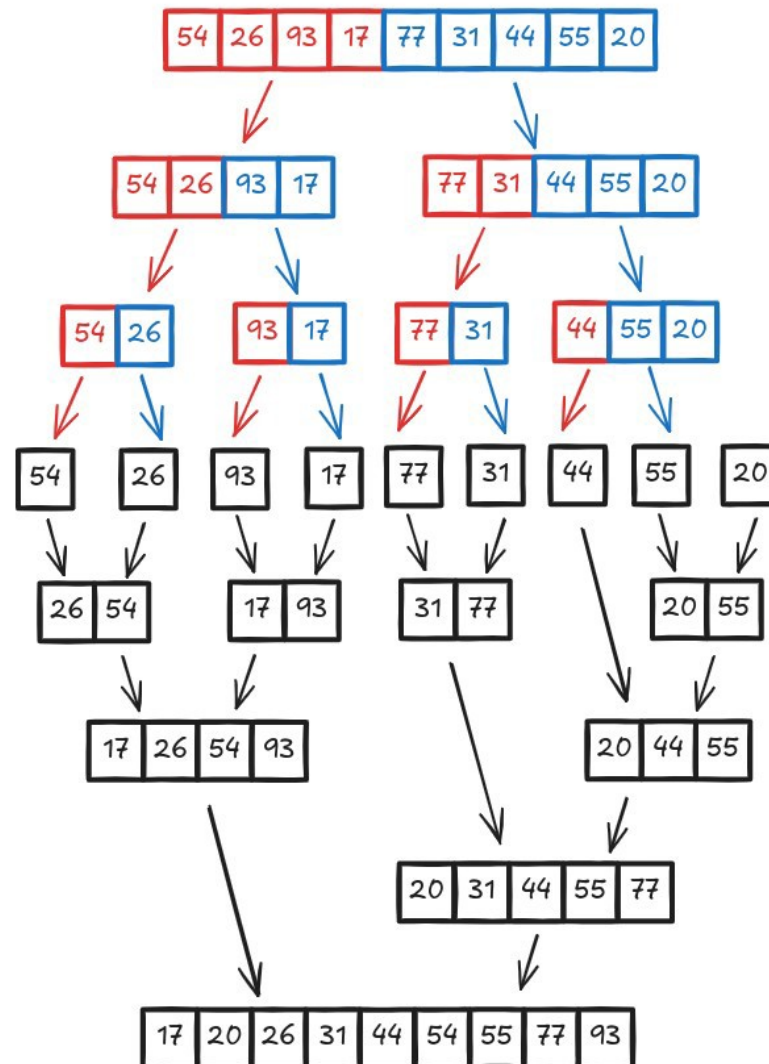
Assim que as metades estiverem ordenadas, a operação fundamental, chamada de intercalação, é realizada. Intercalar é o processo de pegar duas listas menores ordenadas e combiná-las de modo a formar uma lista nova, única e ordenada.

A figura a seguir ilustra as duas fases principais do algoritmo Mergesort: a divisão e a intercalação.

Merge Sort



Merge Sort



Divisão

Intercalação
(Merge)

Merge Sort

Algoritmo:

```
mergeSort( vet, ini, fim ){  
    se( ini >= fim ) retorna;  
    meio = (ini + fim) / 2;  
    mergeSort( vet, ini, meio );  
    mergeSort( vet, meio+1, fim);  
    merge( vet, ini, meio, fim);  
}
```

Aplicar Merge Sort sobre o vetor:

[6 – 5 – 3 – 1 – 8 – 7 – 2 – 4]

Merge Sort

Qual a complexidade de tempo do Merge Sort?

Existe um pior caso? Qual seria?

Qual a complexidade de espaço do Merge Sort?

Merge Sort

É interessante notar que o algoritmo Mergesort se comporta da mesma forma para o caso médio, melhor caso e pior caso.

Os três passos úteis dos algoritmos de dividir para conquistar que se aplicam ao MergeSort são:

1. Dividir: Calcula o ponto médio do sub-arranjo, o que demora um tempo constante $O(1)$;
2. Conquistar: Recursivamente resolve dois subproblemas, cada um de tamanho $n/2$, o que contribui com $T(n/2) + T(n/2)$ para o tempo de execução;
3. Combinar: Unir os sub-arranjos em um único conjunto ordenado, que leva o tempo $O(n)$;

Merge Sort

Assim, podemos escrever a relação de recorrência como:

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n)$$

Para que tenhamos $T(1)$, é preciso que $n = 2k$, onde k é o número de divisões até a partição ter apenas um elemento (o caso base). Ou seja, $k = \log_2 n$. Quando $k = \log_2 n$, temos:

$$T(n) = nT(1) + n \log_2 n$$

Como $T(1)$ é $O(1)$, temos que a complexidade do Mergesort em qualquer caso é **$O(n \log_2 n)$** .

Merge Sort

Uma das maiores limitações do algoritmo MergeSort é que esse método passa por todo o longo processo mesmo se a lista L já estiver ordenada.

Por essa razão, a complexidade de melhor caso é idêntica a complexidade de pior caso, ou seja, $O(n \log n)$.

Para o caso de n muito grande, e listas compostas por números gerados aleatoriamente, pode-se mostrar que o número médio de comparações realizadas pelo algoritmo MergeSort é aproximadamente αn menor que o número de comparações no pior caso

Merge Sort

Qual a complexidade de espaço do Merge Sort?

Como na função merge (não apresentada aqui) necessita de um vetor adicional para unir ambas as sub-listas, a complexidade de espaço deste algoritmo é **$O(n)$** .

Merge Sort - Exercício

Crie o algoritmo Merge, que utiliza é responsável por realizar a intercalação de duas sublistas com custo computacional $O(n)$.

Implemente o método de ordenação MergeSort comparando o tempo de execução para 100, 1000 e 100.000 valores (ordenado crescentemente e decrescentemente e com valores aleatórios).

Quick Sort

A principal ideia do Quick Sort é a ordenação com base em um elemento denominado **pivô**.

Deve-se ordenar o vetor mantendo todos os elementos menores do que o pivô a sua esquerda e todos os elementos maiores do que o pivô a sua direita (**pivoteamento**)

Após este processo realiza-se o mesmo procedimento para o grupo a esquerda do pivô e depois para o grupo a direita do pivô – enquanto o número de elementos for maior do que um.

Quick Sort

Algoritmo:

```
quickSort( vet, ini, fim ){  
    se( ini >= fim ) retorna;  
    meio = pivoteamento( vet, ini, fim );  
    quickSort( vet, ini, meio );  
    quickSort( vet, meio+1, fim );  
}
```

Aplicar Quick Sort sobre o vetor:

[6 – 5 – 3 – 1 – 8 – 7 – 2 – 4]

Quick Sort

O algoritmo pode ser dividido em 3 passos principais:

1. Escolha do pivô:

- a) Primeiro elemento da lista
- b) Último elemento da lista
- c) Mediana entre primeiro, central e último elementos (melhor)

2. Particionamento: reorganizar o vetor de modo que todos os elementos menores que o pivô apareçam antes dele (a esquerda) e os elementos maiores apareçam após ele (a direita). Ao término dessa etapa o pivô estará em sua posição final (existem várias formas de se fazer essa etapa)

3. Ordenação: recursivamente aplicar os passos acima aos sub-vetores produzidos durante o particionamento. O caso limite da recursão é o sub-vetor de tamanho 1, que já está ordenado.

Quick Sort

O Quicksort é um algoritmo de ordenação que usa o método de dividir para conquistar, e o particionamento é a etapa central, com dois métodos populares: Lomuto e Hoare.

Lomuto é mais simples, geralmente escolhendo o último elemento como pivô e organizando o array para que todos os menores ou iguais fiquem à esquerda do pivô e os maiores à direita, retornando a posição final do pivô. Porém, tem um desempenho pior em arrays já ordenados ou com muitos elementos iguais.

Hoare é mais eficiente na prática, mas mais complexo, e não garante que o pivô termine na sua posição final, trocando elementos desalojados de ambas as pontas do array até que os ponteiros se cruzem. É o método preferido na prática para uma melhor performance geral do Quicksort.

Quick Sort

Algoritmo de particionamento Lomuto:

```
lomuto( vet, ini, fim ){  
    pivo = ini;  
    para( j=ini+1; j<=fim; j++ ){  
        se( vet[j] < vet[ini] ){  
            pivo++;  
            troca( vet[pivo], vet[j] );  
        }  
    }  
    troca( vet[ini], vet[pivo] );  
    retorne pivo;  
}
```

Quick Sort

Algoritmo de particionamento Hoare:

```
hoare( vet, ini, fim ){  
    pivo = vet[ini];  
    i = ini;  
    j = fim;  
    repita{  
        enquanto( vet[i] < pivo ) faça i = i+1;  
        enquanto( vet[j] > pivo ) faça j = j-1;  
        se( i < j ) troca( vet[i] , vet[j] );  
        senão retorne j;  
    }  
}
```

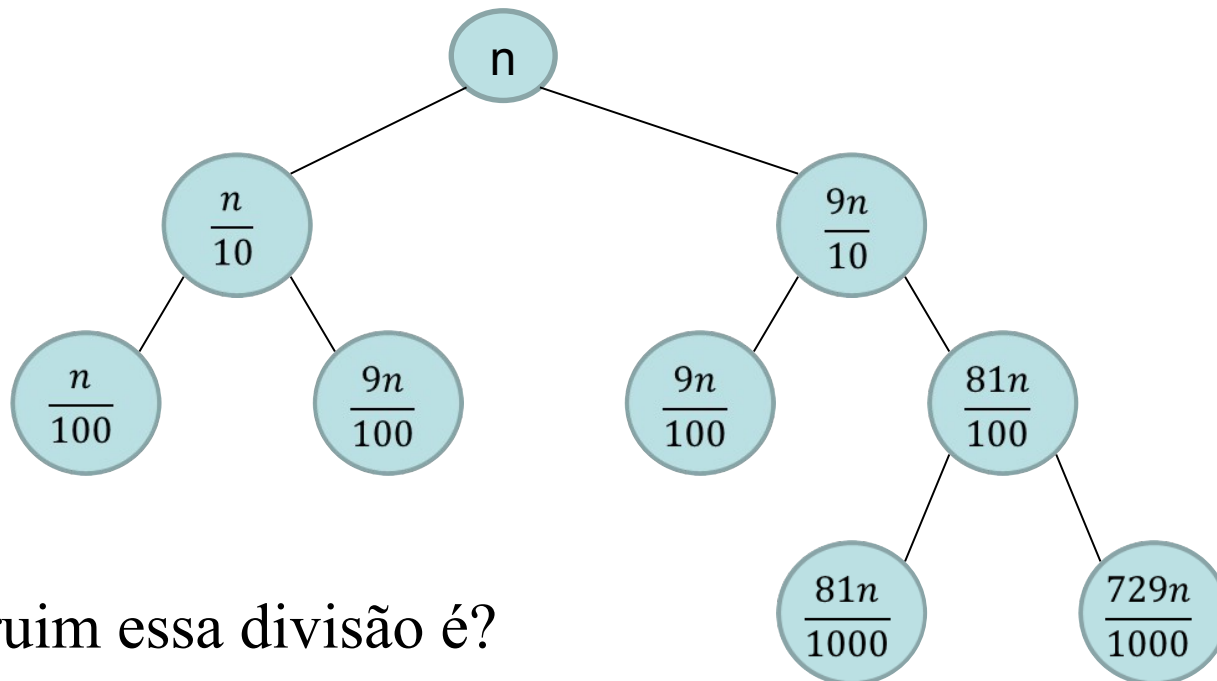
Quick Sort

Perguntas:

- Qualquer pivô serve?
- Existe um pivô ruim ou bom?
- Como escolher um pivô adequado?
- Qual o melhor caso para o Quick Sort? (complexidade)
- Qual o pior caso para o Quick Sort? (complexidade)
- Qual a complexidade de espaço do Quick Sort?

Quick Sort

Considere o seguinte cenário onde ocorre uma divisão desbalanceada de proporção constante 1:9



Quão ruim essa divisão é?

Quick Sort

Quão ruim essa divisão é?

$$T(n) = T(n/10) + T(9n/10) + n$$

$$T(1) = 1$$

(resolva a recursão com alguns valores arbitrários para **n** e compare o crescimento com as funções n^2 e $n \log n$)

Ordenação em tempo Linear

Counting Sort + Bucket Sort

Ordenações Lineares

Métodos de ordenação por comparação:

$$\Omega(n \log n)$$

Ordenações lineares [$\Omega(n)$] só são possíveis em **determinadas** condições.

Ordenações Lineares

Nem todos os algoritmos de ordenação são baseados em comparações.

Existem diversos métodos que ordenam listas sem a necessidade de comparar e trocar elementos.

Alguns exemplos são: CountingSort, BucketSort, RadixSort e PigeonHoleSort.

Veremos a seguir os algoritmos CountingSort e BucketSort

Counting Sort

O algoritmo Countingsort ordena os elementos de uma sequência pela contagem do número de ocorrências de cada elemento. Para mostrar a intuição por trás do algoritmo, vamos analisar seus passos:

Passo 1. Encontrar o maior elemento da lista.

Passo 2. Criar um vetor de tamanho $(\text{max}+1)$ composto por zeros.

Passo 3. Armazenar o número de ocorrências de cada elemento em seu respectivo índice em H.

Passo 4. Calcule a soma cumulativa dos elementos de H.

Passo 5. Associar os índices de cada elemento das listas.

Passo 6. Reduzir os valores associados anteriormente.

Counting Sort - exemplo

Passo 1. Encontrar o maior elemento da lista.

$L = [4, 2, 2, 8, 3, 3, 1]$

Counting Sort - exemplo

Passo 1. Encontrar o maior elemento da lista.

$L = [4, 2, 2, 8, 3, 3, 1]$ $\max = 8$

Counting Sort - exemplo

Passo 1. Encontrar o maior elemento da lista.

$L = [4, 2, 2, 8, 3, 3, 1]$ $\max = 8$

Passo 2. Criar um vetor de tamanho $(\max + 1)$ composto por zeros.

$H = [0, 0, 0, 0, 0, 0, 0, 0, 0]$

Counting Sort - exemplo

$L = [4, 2, 2, 8, 3, 3, 1]$ $\max = 8$

Passo 3. Armazene o número de ocorrências de cada elemento em seu respectivo índice em H. Note que poderão sobrar vários elementos nulos em H (não existem ocorrências para 0, 5, 6 e 7).

$H = [0, 1, 2, 2, 1, 0, 0, 0, 1]$

Counting Sort - exemplo

$L = [4, 2, 2, 8, 3, 3, 1] \text{ max} = 8$

$H = [0, 1, 2, 2, 1, 0, 0, 0, 1]$

Passo 4. Fazer a soma cumulativa dos elementos de H.

$C = [0, 1, 3, 5, 6, 6, 6, 6, 7]$

Counting Sort - exemplo

$L = [4, 2, 2, 8, 3, 3, 1]$ $\max = 8$

$H = [0, 1, 2, 2, 1, 0, 0, 0, 1]$

$C = [0, 1, 3, 5, 6, 6, 6, 6, 7]$

Passo 5. Encontre o índice de cada elemento da lista L no vetor C , e coloque o respectivo elemento no índice $C[L[i]] - 1$ de uma lista S com o mesmo número de elementos de L .

$S = [\quad , \quad , \quad , \quad , \quad , \quad , \quad]$

Counting Sort - exemplo

$L = [4, 2, 2, 8, 3, 3, 1]$ $\max = 8$

$H = [0, 1, 2, 2, 1, 0, 0, 0, 1]$

$C = [0, 1, 3, 5, 6, 6, 6, 6, 7]$

Passo 5. Encontre o índice de cada elemento da lista L no vetor C , e coloque o respectivo elemento no índice $C[L[i]] - 1$ de uma lista S com o mesmo número de elementos de L .

$$C[L[0]] - 1 = C[4] - 1 = 6 - 1 = 5$$

$S = [\quad , \quad , \quad , \quad , \quad , \quad]$

Counting Sort - exemplo

$L = [4, 2, 2, 8, 3, 3, 1]$ $\max = 8$

$H = [0, 1, 2, 2, 1, 0, 0, 0, 1]$

$C = [0, 1, 3, 5, 6, 6, 6, 6, 7]$

Passo 5. Encontre o índice de cada elemento da lista L no vetor C , e coloque o respectivo elemento no índice $C[L[i]] - 1$ de uma lista S com o mesmo número de elementos de L .

$$C[L[0]] - 1 = C[4] - 1 = 6 - 1 = 5 \Rightarrow S[5] = 4$$

$S = [, , , , , \mathbf{4} ,]$

Counting Sort - exemplo

$L = [4, 2, 2, 8, 3, 3, 1]$ $\max = 8$

$H = [0, 1, 2, 2, 1, 0, 0, 0, 1]$

$C = [0, 1, 3, 5, 6, 6, 6, 6, 7]$

Passo 5. Encontre o índice de cada elemento da lista L no vetor C , e coloque o respectivo elemento no índice $C[L[i]] - 1$ de uma lista S com o mesmo número de elementos de L .

$$C[L[1]] - 1 = C[2] - 1 = 3 - 1 = 2 \Rightarrow S[2] = 2$$

$S = [, , \mathbf{2} , , , 4 ,]$

Counting Sort - exemplo

$L = [4, 2, 2, 8, 3, 3, 1]$ $\max = 8$

$H = [0, 1, 2, 2, 1, 0, 0, 0, 1]$

$C = [0, 1, 3, 5, 6, 6, 6, 6, 7]$

Passo 6. Após colocar cada elemento na sua posição correta, diminua seu valor associado em C de uma unidade.

$S = [1, 2, 2, 3, 3, 4, 8]$

Counting Sort

```
Countingsort(L, n) {  
    m = max(L)          # maior elemento de L  
    C = zeros(m+1)      # vetor de 0 a m  
    S = zeros(n)        # vetor com mesmo tamanho de L  
    for i = 0 to n - 1  
        C[L[i]] = C[L[i]] + 1 # conta número de ocorrências  
    for i = 1 to m  
        C[i] = C[i] + C[i-1] # soma acumulada  
    for i = 0 to n - 1 {  
        C[L[i]] = C[L[i]] - 1  
        S[C[L[i]]] = L[i]  
    }  
    return S  
}
```

Counting Sort

Primeiramente, note que encontrar o maior elemento de L possui complexidade $O(n)$. Analisando cada uma das 3 estruturas de repetição (*for*), podemos escrever a função $T(n)$ como:

$$T(n) = O(n) + O(n) + O(m) + O(n) = \mathbf{O(n+m)}$$

Em geral, o algoritmo funciona muito bem quando o maior elemento da lista não é tão grande. Porém, esse algoritmo tem limitações. A principal delas é quando o maior elemento é muito grande e isso faz com que o vetor S seja muito maior que L .

Counting Sort

Pressupõe valores inteiros no intervalo 1 a k.

Algoritmo:

- contar o nº de elementos menores que 'x';
- usar esta informação para alocar o elemento na sua posição correta no vetor final;

Exemplos:

Caso bom

2	5	3	0	2	3	0	5	3	0
---	---	---	---	---	---	---	---	---	---

Caso ruim

2	25	13	100	250	300.000	1	5	3	0
---	----	----	-----	-----	---------	---	---	---	---

Bucket Sort

Também não é baseado em comparações, por isso não precisa realizar trocas.

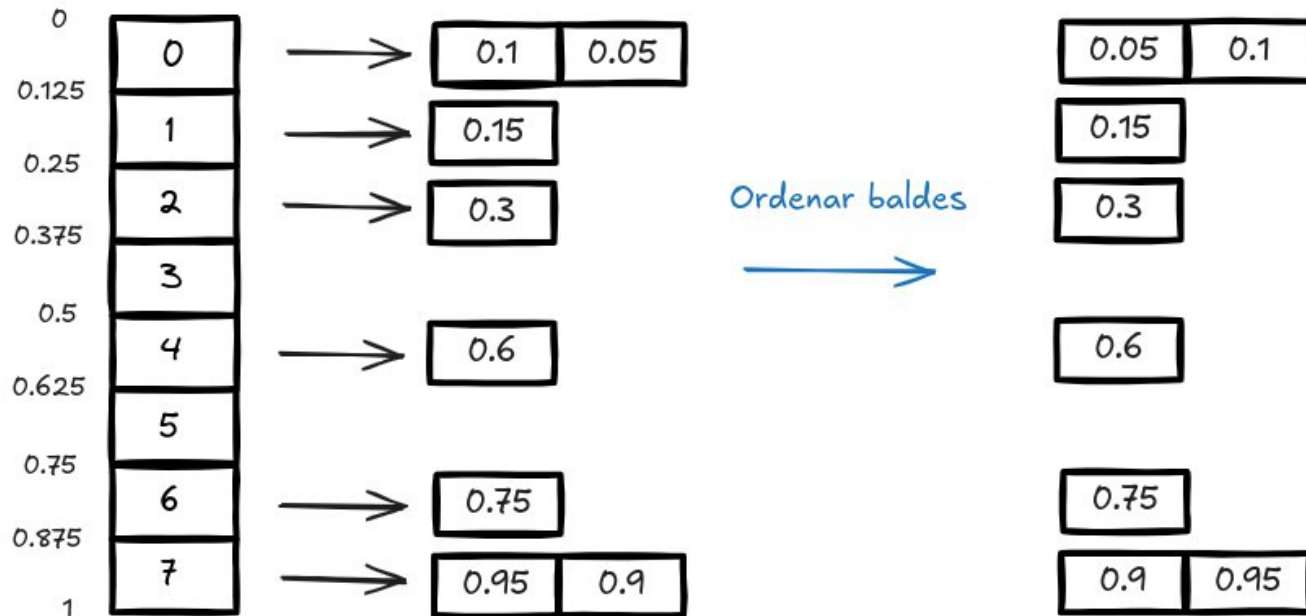
Seja $I = [0, K)$ um intervalo arbitrário, o primeiro passo do algoritmo consiste em dividir o intervalo de tamanho K em n baldes (buckets), cada um com tamanho K/n .

A ideia é simples: devemos colocar cada número da lista L a ser ordenada em seu respectivo balde. Sob a hipótese de uniformidade, o número de elementos por balde será pequeno e constante, fazendo com que o custo computacional da ordenação de cada balde seja $O(1)$ (já que n é muito pequeno).

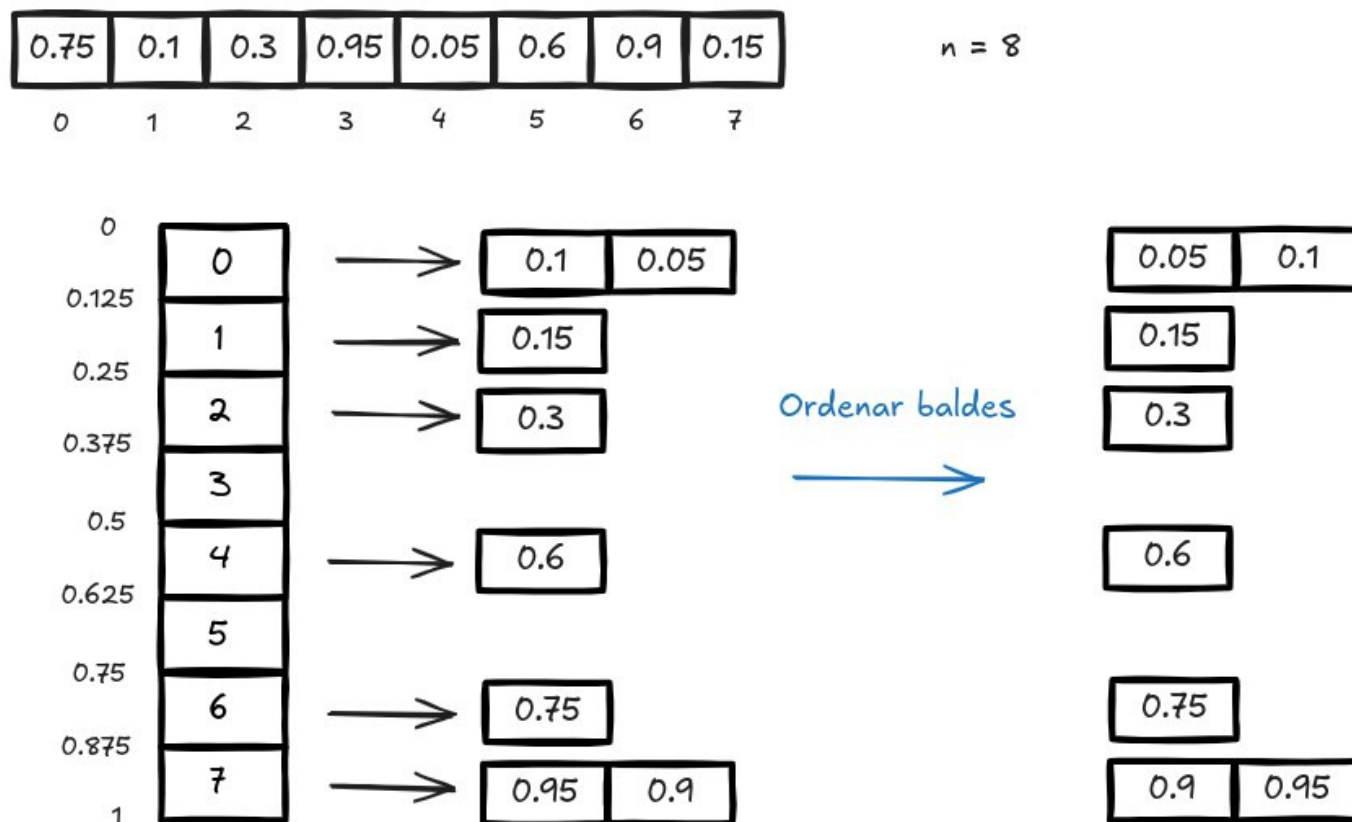
Bucket Sort

0.75	0.1	0.3	0.95	0.05	0.6	0.9	0.15
0	1	2	3	4	5	6	7

$n = 8$



Bucket Sort

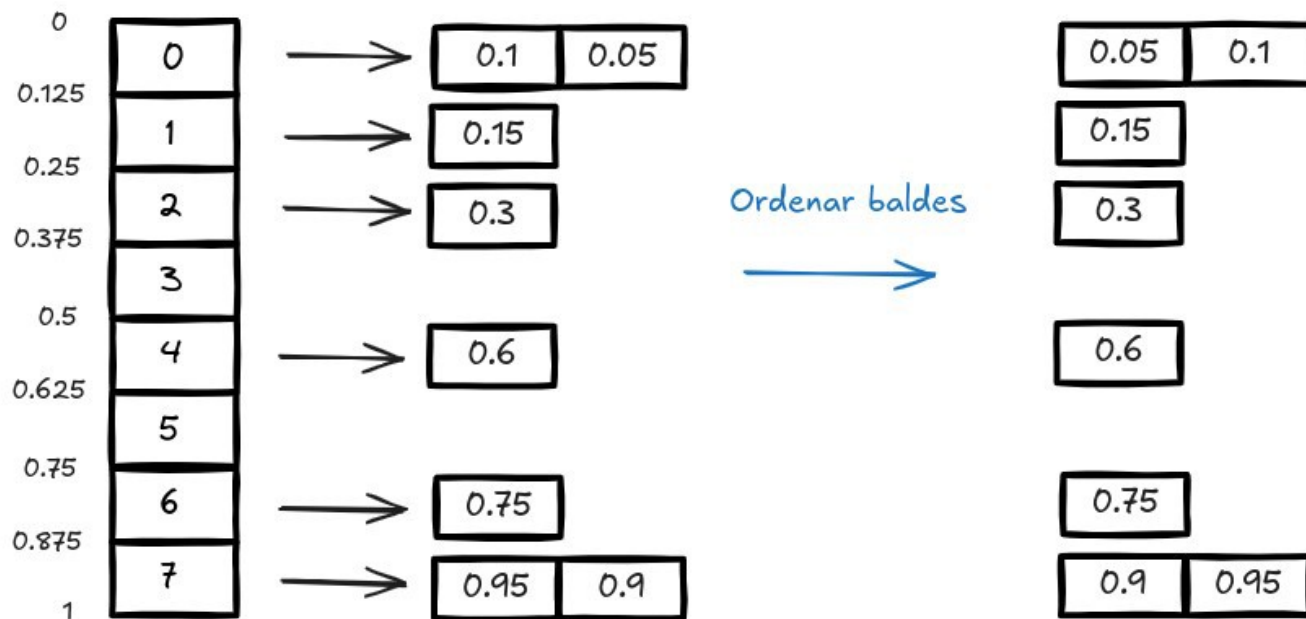


Por fim, percorremos os baldes na ordem em que aparecem copiando os elementos ordenados de cada balde para a lista final.

Bucket Sort

0.75	0.1	0.3	0.95	0.05	0.6	0.9	0.15
0	1	2	3	4	5	6	7

$n = 8$



Ordenar baldes



0.05	0.1	0.15	0.3	0.6	0.75	0.9	0.95
0	1	2	3	4	5	6	7

coletar baldes



Bucket Sort

```
BucketSort(L, n) {  
    for i = 0 to n - 1  
        B[i] = NIL      # Cria n baldes vazios  
    for i = 0 to n - 1 {  
        if L[i] == 1  
            index = n - 1  
        else {  
            index = floor(n*L[i])  
            insert(B[index], L[i])  
        }  
    }  
    for i = 0 to n - 1  
        sort(B[i])  
    S = concatenate(B[0], B[1], ..., B[n-1])  
    return S  
}
```

Bucket Sort

Note que utilizamos 3 primitivas básicas no algoritmo Bucketsort.

i) $\text{insert}(p, x)$: insere x na lista apontada pela referência p . Possui complexidade $\mathbf{O(1)}$.

ii) $\text{sort}(p)$: ordena a lista apontada pela referência p . Complexidade seria entre $O(n \log n)$ e $O(n^2)$.

iii) $\text{concatenate}(B, n)$: retorna a lista obtida pela concatenação das listas $B[0], B[1], \dots, B[n-1]$. Complexidade $O(kn)$, onde k é uma constante que representa o tamanho médio dos baldes, o que resulta em $\mathbf{O(n)}$.

Bucket Sort

O ponto crítico é a análise da primitiva `sort()`, sendo que devemos realizar uma análise probabilística. O número médio de comparações para ordenar o balde $B[i]$ é $2n-1$ o que indica complexidade **$O(n)$** . Por essa razão, a complexidade total do Bucke Sort é:

$$T(n) = n * O(1) + O(n) + O(n) = \mathbf{O(n+m)}$$

o que finalmente resulta em **$O(n)$** . Vimos que o Bucketsort é um algoritmo de ordenação linear no caso médio. Mas e no pior caso? Esse cenário ocorre quando todos os elementos caem no mesmo balde. Sendo assim, se optarmos pela aplicação de um algoritmo baseado em trocas, a complexidade poderia variar de $O(n \log n)$ a $O(n^2)$.

Bucket Sort

Pressupõe que a entrada consiste de elementos com distribuição de valores uniforme.

Algoritmo:

- separar os elementos em grupos / baldes;
- ordenar os elementos nos seus baldes;

Exemplos:

Caso bom

20	16	43	0	22	13	33	40	31	7
----	----	----	---	----	----	----	----	----	---

Caso ruim

2	43	3	5	0	7	8	1	6	4
---	----	---	---	---	---	---	---	---	---

Comparação de tempos para ordenação

Algoritmo	Melhor	Médio	Pior
BubbleSort	$O(n)$	$O(n^2)$	$O(n^2)$
InsertionSort	$O(n)$	$O(n^2)$	$O(n^2)$
SelectionSort	$O(n^2)$	$O(n^2)$	$O(n^2)$
ShellSort	$O(n \log n)$	$O(n^{1.25})$ a $O(n \log^2 n)$	$O(n \log^2 n)$ a $O(n^{1.333})$
MergeSort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
QuickSort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$
Heapsort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
Countingsort	$O(n+m)$	$O(n+m)$	$O(n+m)$
Radixsort	$O(nd)$	$O(nd)$	$O(nd)$
Bucketsort	$O(n)$	$O(n)$	$O(n \log n)$

Exercício

Defina a complexidade de espaço para cada um dos seguintes algoritmos de ordenação.

Algoritmo	Espaço
BubbleSort	
InsertionSort	
SelectionSort	
ShellSort	
MergeSort	
QuickSort	
Heapsort	
Countingsort	
Radixsort	
Bucketsort	

Exercício

Defina a complexidade de espaço para cada um dos seguintes algoritmos de ordenação.

Algoritmo	Espaço
BubbleSort	$O(1)$
InsertionSort	$O(1)$
SelectionSort	$O(1)$
ShellSort	$O(1)$
MergeSort	$O(n)$
QuickSort	$O(\log n)$ (pior caso $O(n)$)
Heapsort	$O(1)$
Countingsort	Varia (potencialmente $O(n)$)
Radixsort	Varia (potencialmente $O(n+k)$)
Bucketsort	$O(n)$

Exercício de implementação

- Implemente um dos algoritmos de ordenação desta aula e compare o tempo de execução com aqueles implementados anteriormente. Faça um quadro comparativo do tempo de execução para ordenar:
 - (n=)100.000 números aleatórios
(OBS: utilize a mesma entrada para cada algoritmo)
 - (n=)100.000 números já em ordem crescente